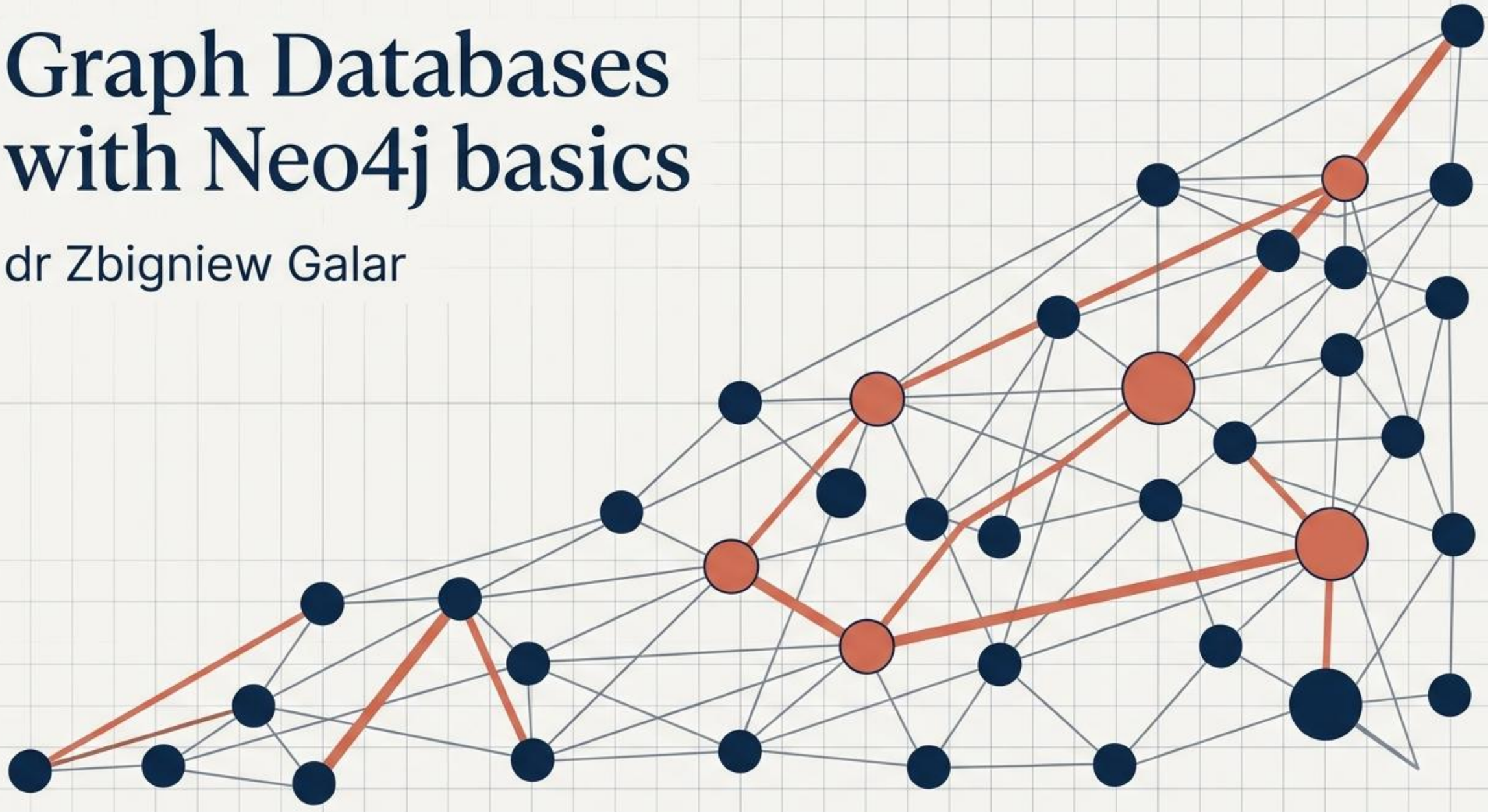
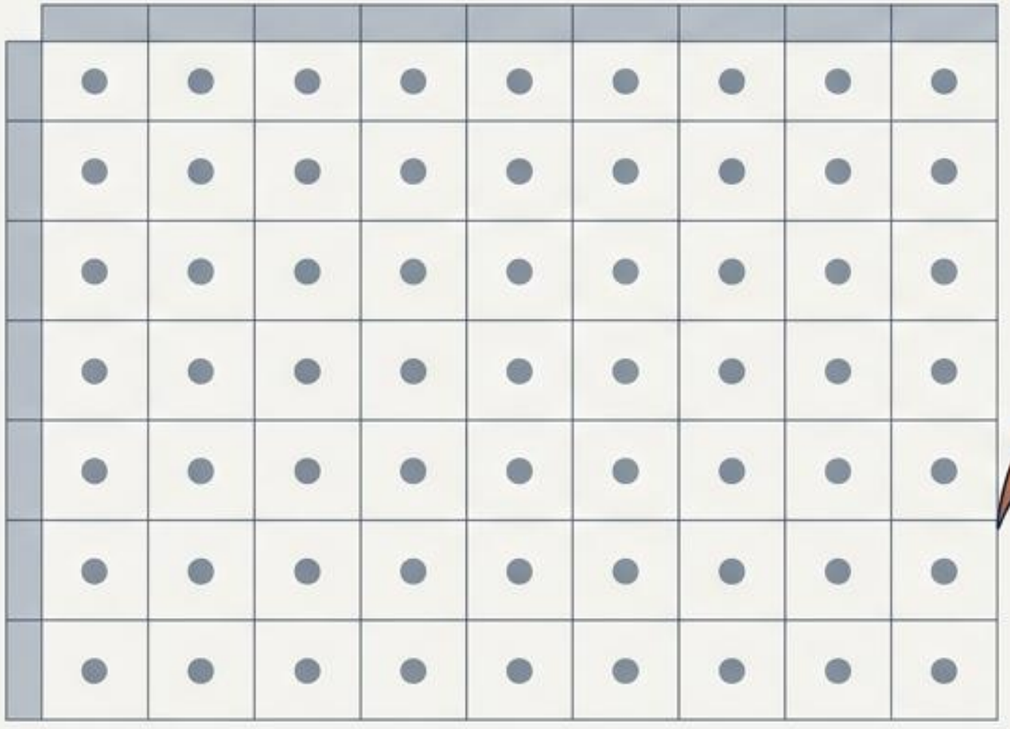
Graph Databases with Neo4j basics

dr Zbigniew Galar

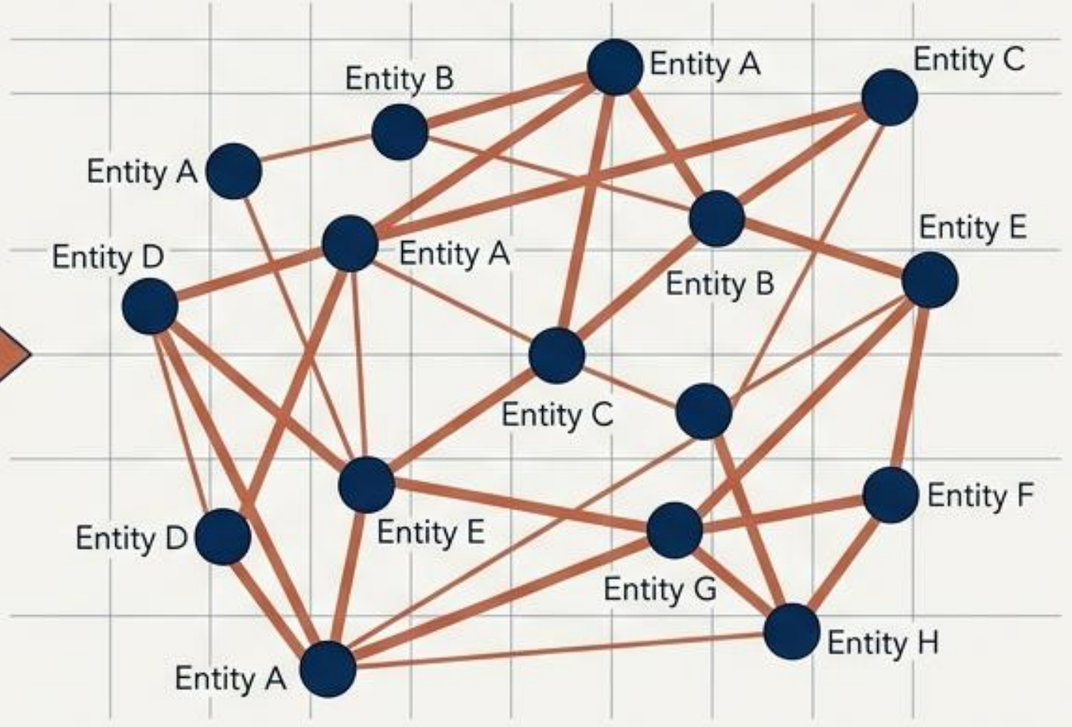


Connections dictate the value of your data

The Traditional View



The Graph View

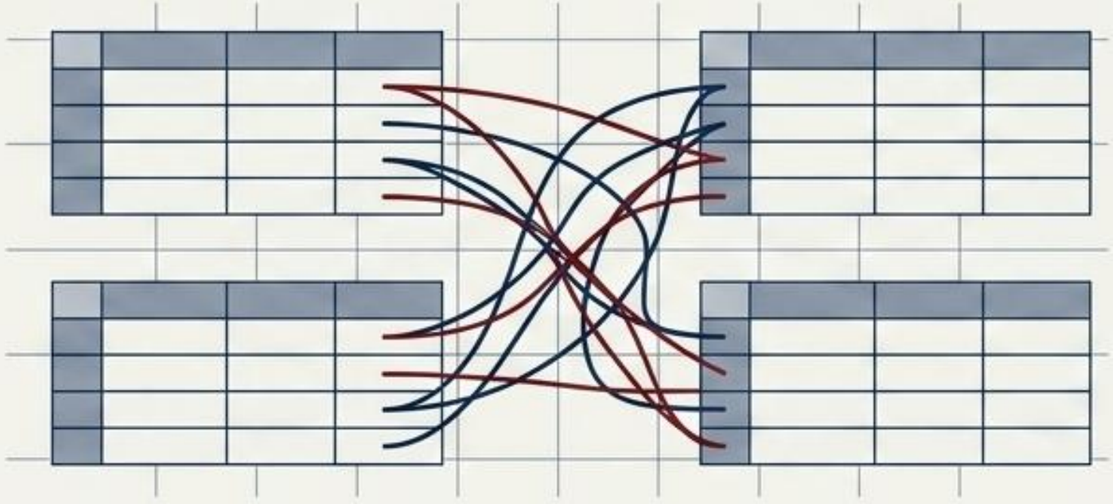


Connections between the data are exactly as important as the data itself.

A graph is a set of objects related to each other. Vertices represent the entities; edges represent the precise relationships between them.

Relational rigidity versus graph agility

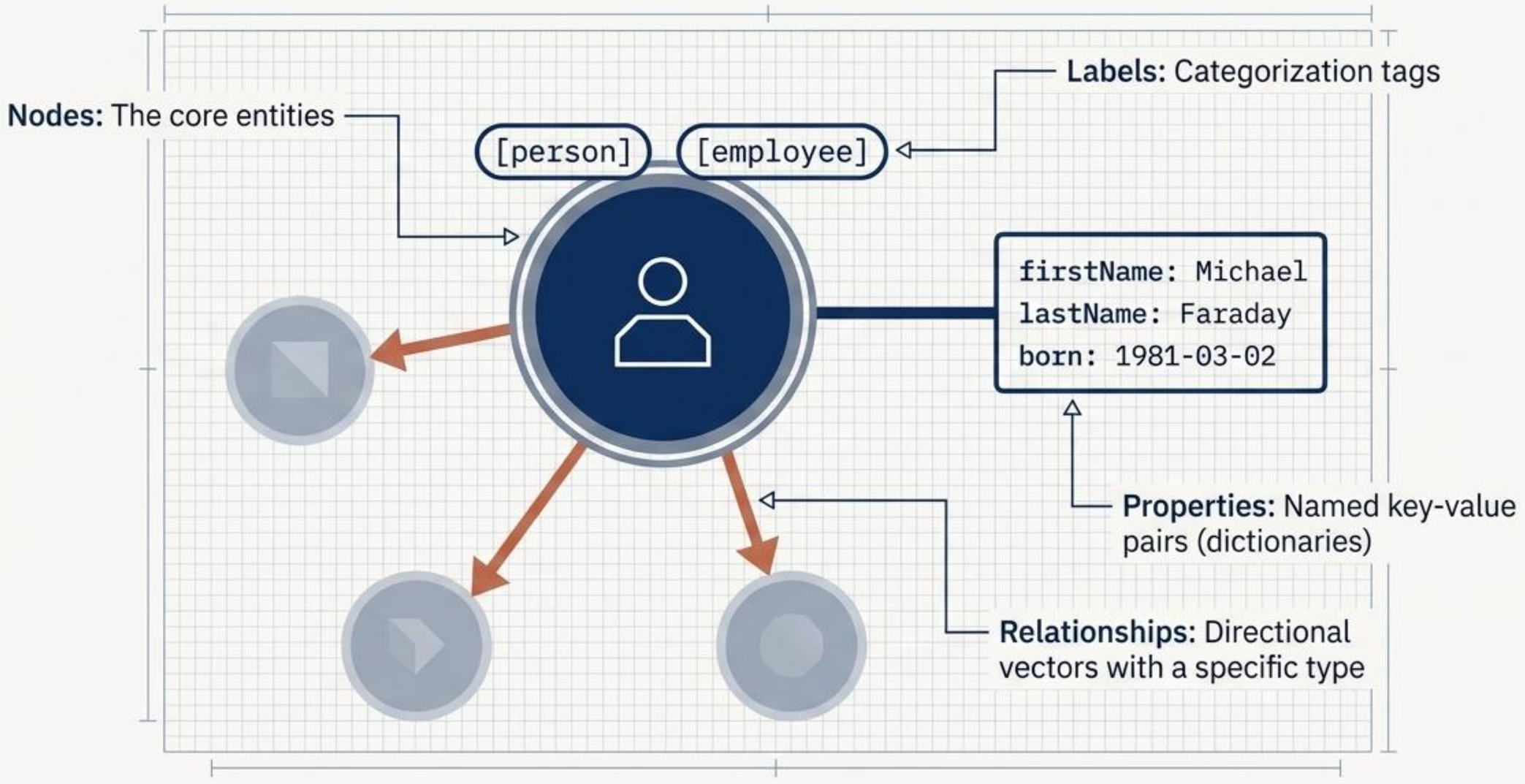
The Join Problem



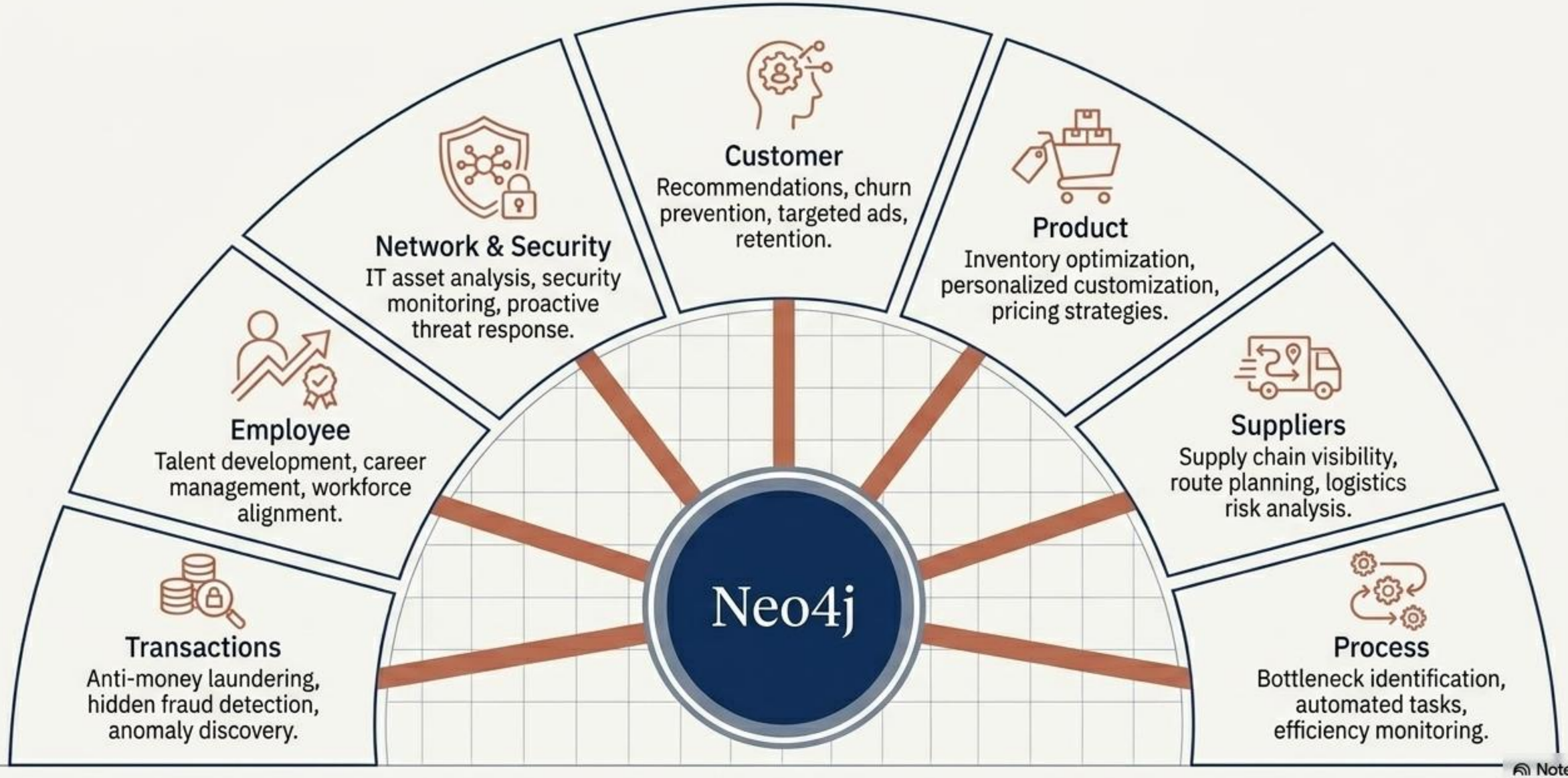
The Graph Hop

Dimension	Relational Database	Graph Database (Neo4j)
Hierarchy Handling	Forced fits into flat rows; highly inefficient for deep trees.	Native, deep hierarchy support. Data is naturally stored as relationships.
Query Speed at Scale	Joins compute at read-time. Index inflates as data grows, response times plummet.	Latency grows strictly with the number of nodes fetched, ignoring the volume of unconnected data.

Anatomy of a labeled property graph



Uncovering patterns across the enterprise

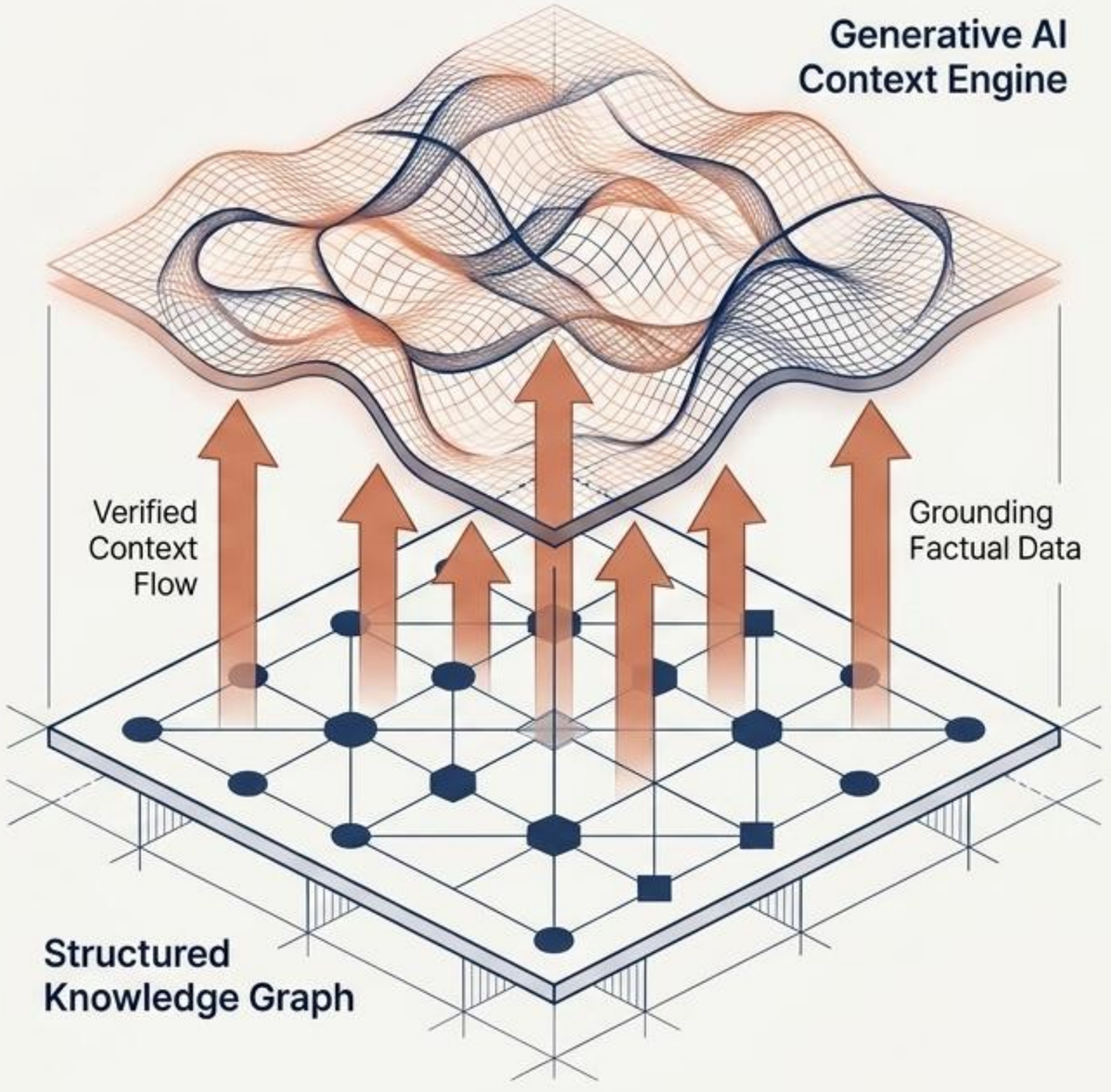


Knowledge graphs ground Generative AI in reality

The Context Gap: GenAI applications are fundamentally dependent on accessing the actual meaning within data.

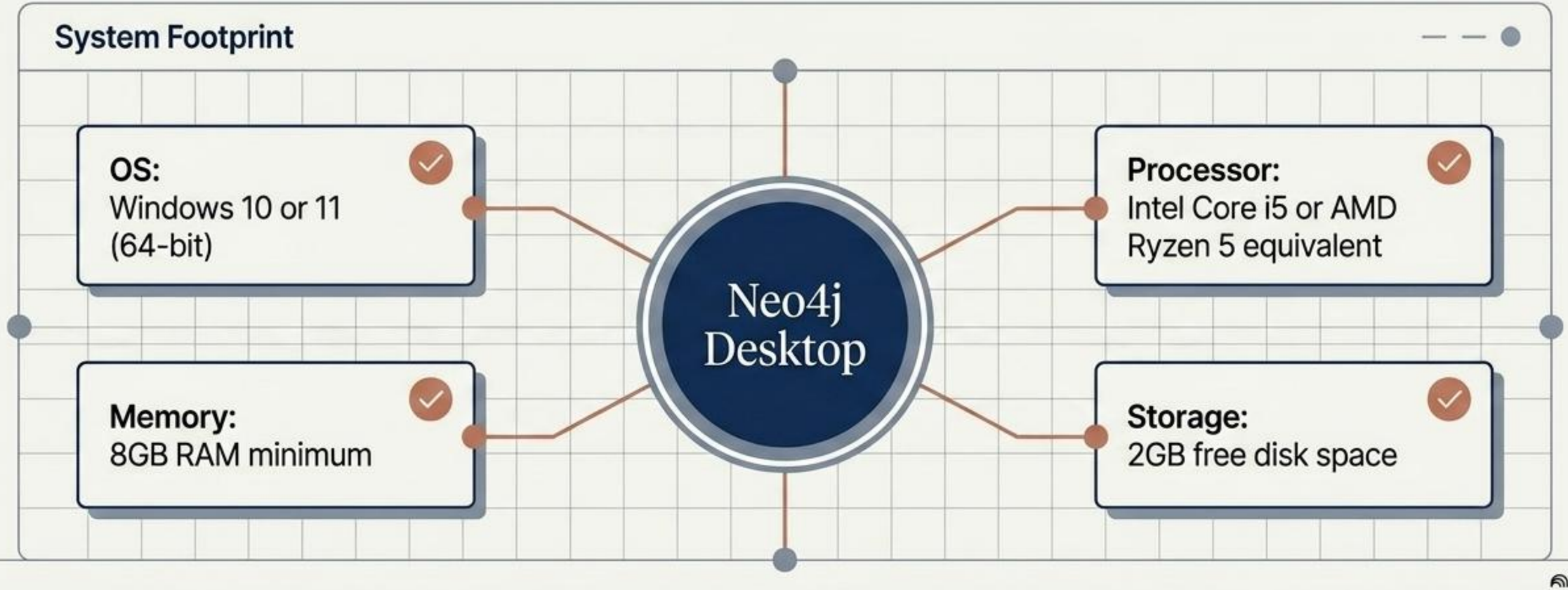
The Integration Engine: Knowledge graphs break down isolated information silos, providing a structured, interconnected representation of entities and attributes.

Real-World Application: Major search engines rely on this exact architecture to map out people, places, and things to prevent hallucinations and supply verified context.



Preparing the Neo4j deployment environment

The Deployment Model: Neo4j Desktop is the standard deployment for analytics. It bundles the graph engine, management UI, and visualization tools, eliminating the need for command-line configuration.



Deployment phase one: Download and scope

1. Download & Version Heuristic

Rule: Always select the latest stable release (5.x).

Rejection: Actively reject pre-release (beta) and legacy (4.x) versions.

Action: Register with email to retrieve the executable and activation key.

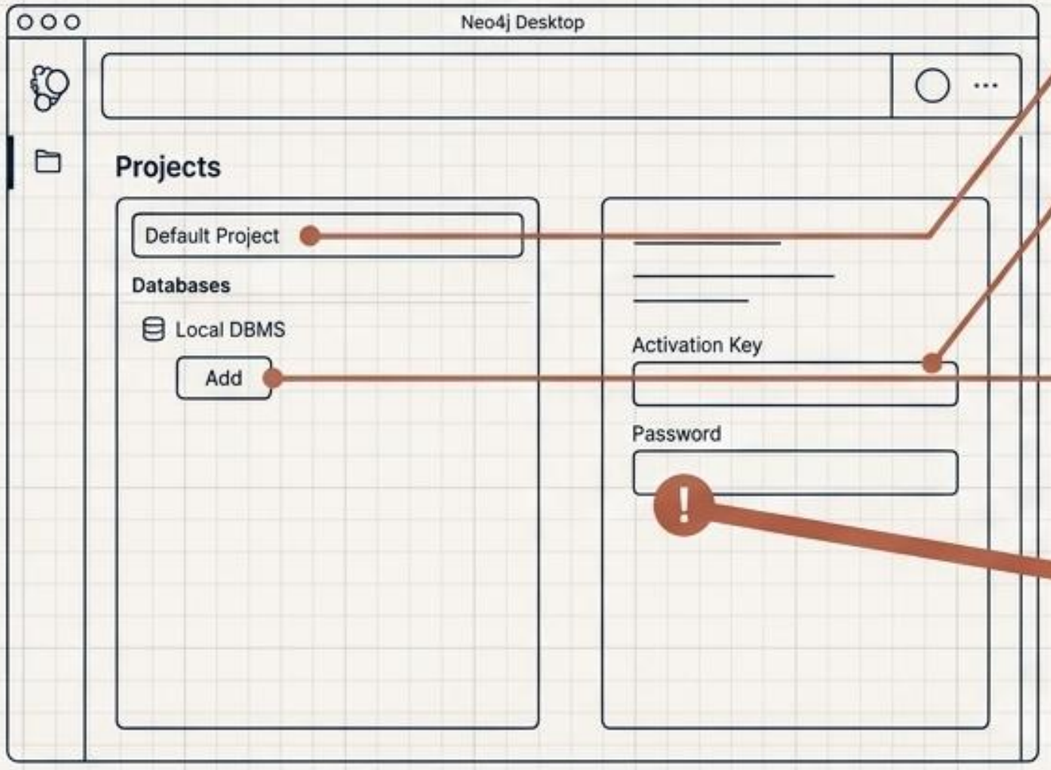
2. Execute & Scope Heuristic

- **Shared Lab Computer** → Install for all users
- **Personal Laptop** → Install for me only

3. Destination Target

`C:\Users\<Username>\AppData\Local\Programs\Neo4j Desktop`

Deployment phase two: Engine configuration



1. **Software Activation:** Paste the activation key.

2. **Project Initialization:** Rename default project to match analysis goal.

3. **Database Creation:** Click Add -> Local DBMS. Name the database (e.g., SupplyChainGraph).

4. **The Password Heuristic**

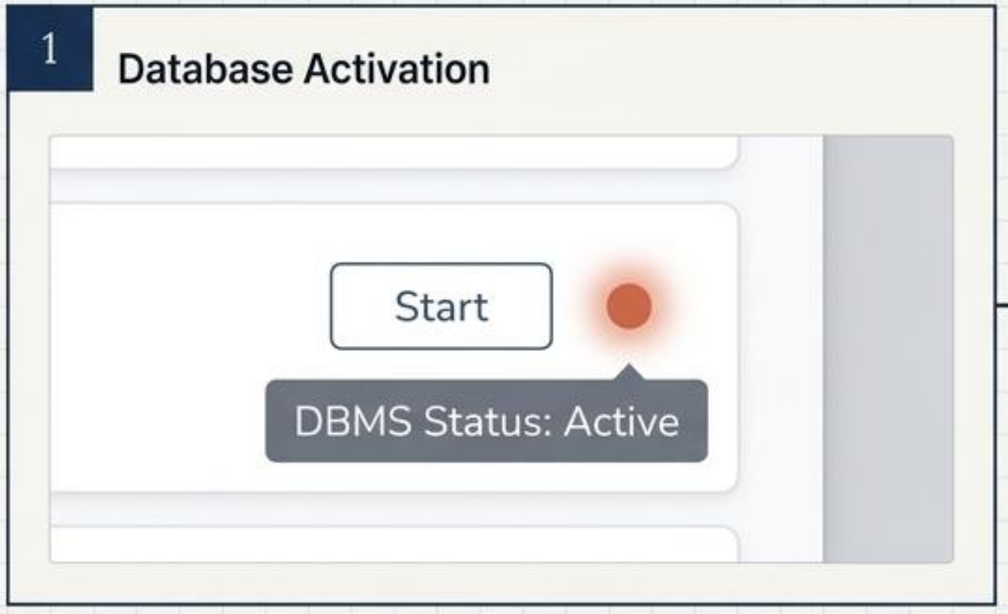
   Rule: Use a standard alphanumeric string (e.g., GraphAdmin123).

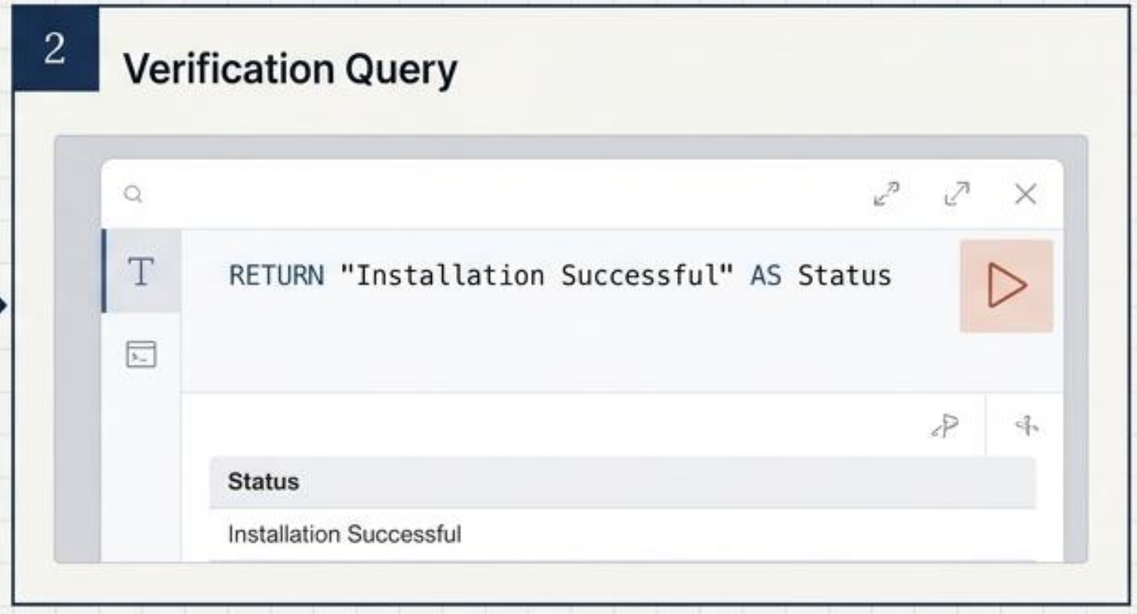
   Rejection: Never use special characters (@, #, %). This prevents fatal URL-encoding errors when connecting external BI tools.

Deployment phase three: Verification and execution

1. **Ignite the Engine:** Hover over the newly created DBMS and click **Start**. Wait for the status indicator to turn active.

2. **Access the Interface:** Click **Open** to launch the Neo4j Browser.





3. **Execute First Query:** Type the following Cypher command and press **play**:

```
RETURN "Installation Successful" AS Status
```

4. **Verify:** A results table will populate, confirming successful execution against the active graph engine.